Builder Pattern

Constructing complex objects step by step with precision.

Presentation Outline

01
The Definition

What is the Builder design pattern?

02
Why Use It?

The core problem it solves in code.

03
Class Diagram

Architectural structure in C#.

04
Before & After

Real-world C# implementation.

05
Pros & Cons

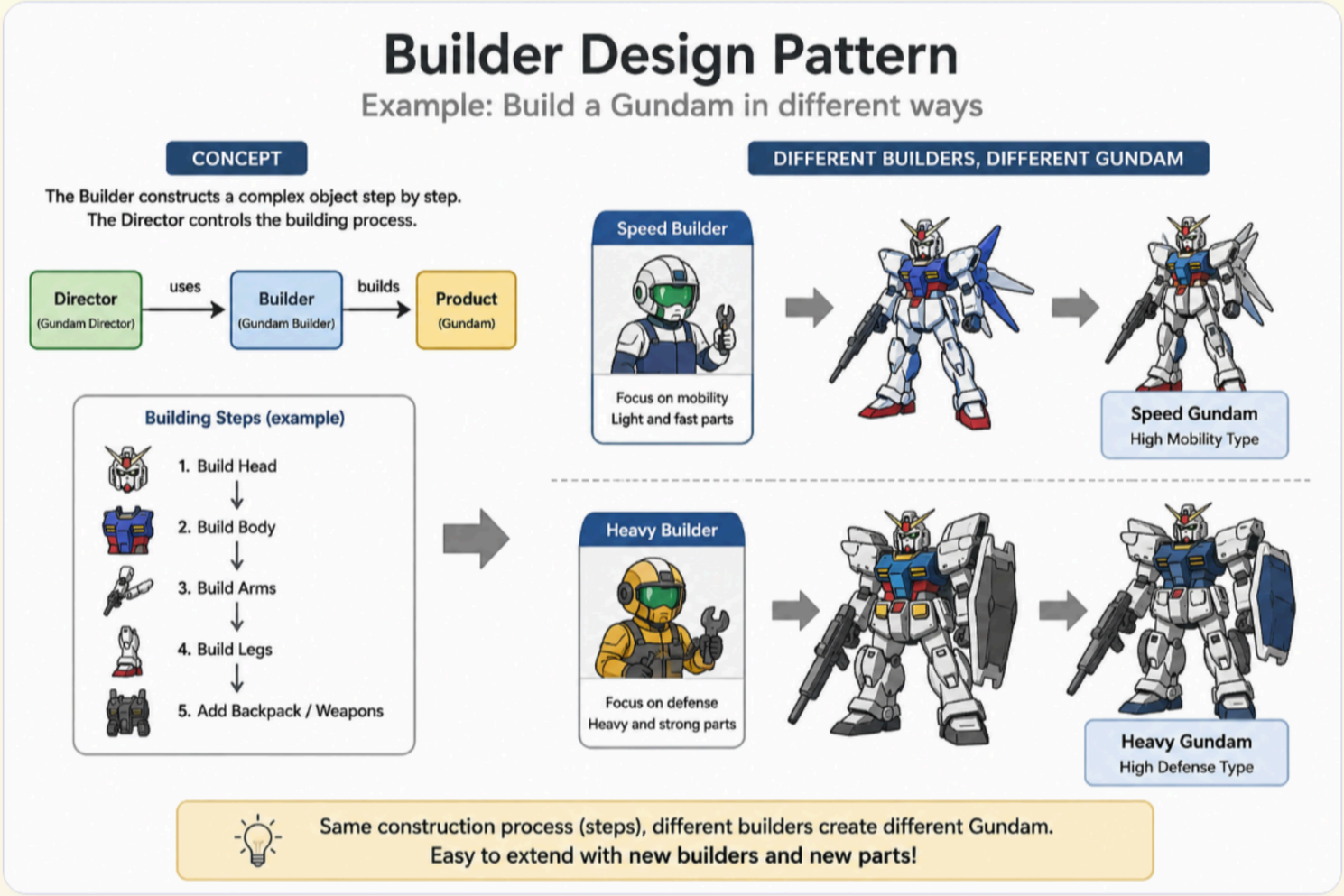
Trade-offs to consider.

What is the Builder Pattern?

Builder is a creational design pattern that lets you construct complex objects step by step.

It extracts the object construction code out of its own class and moves it to separate objects called *builders*. This pattern allows you to produce different types and representations of an object using the same construction code.

The Builder Concept



The Telescoping Constructor Problem

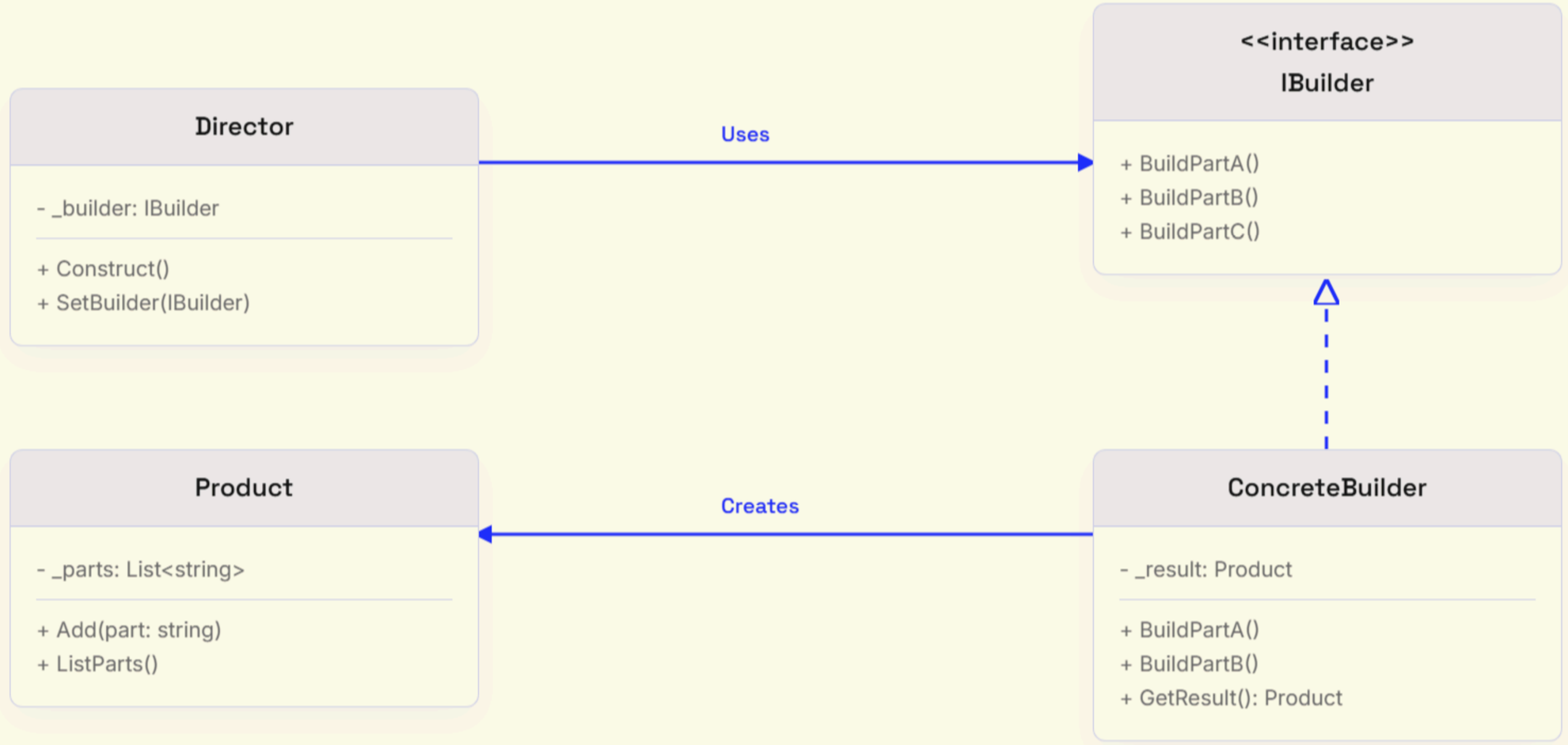
The Problem

- Massive constructors with dozens of parameters.
- Most parameters are unused, leading to ugly code like `new House(4, 2, true, false, null)`.
- Extremely difficult to remember the order of parameters.
- Creating subclasses for every possible configuration causes an explosion of classes.

The Solution

- Extract construction logic into dedicated **Builder** classes.
- Construct objects dynamically using clean, fluent interfaces.
- Only execute the specific steps you actually need.
- Defer final instantiation until the entire configuration is ready.

C# Class Diagram



The Anti-Pattern

Creating a complex object requires specifying every possible detail in a massive constructor call, often passing many `null` or `false` values which makes the code brittle.

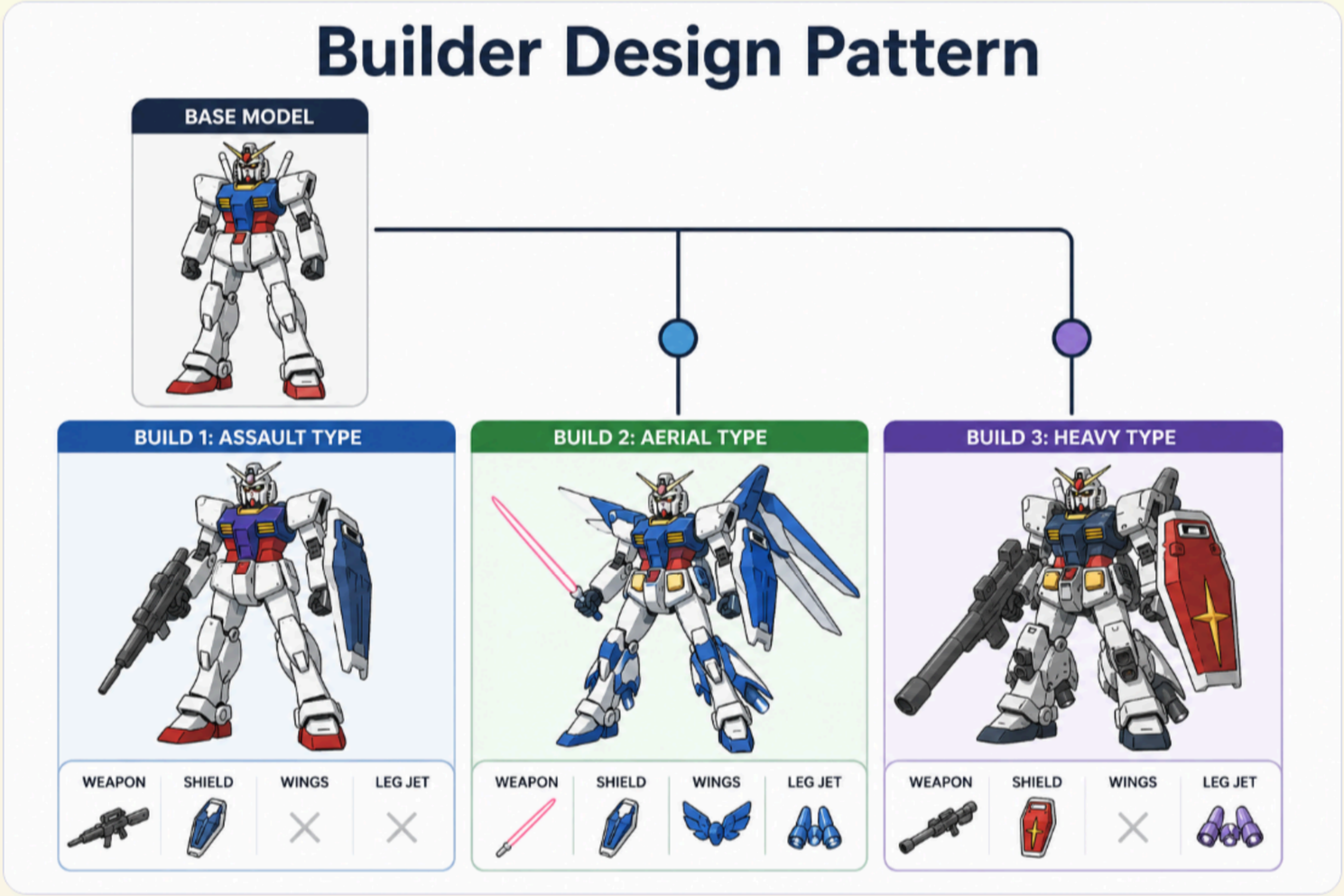
```
// Hard to read, easy to mess up parameter order, completely rigid.
Gundam heavyGundam = new Gundam(
    "Heavy Sensor Head",      // head
    "Heavy Armor Body",       // body
    "Heavy Arms",             // arms
    "Reinforced Legs",        // legs
    "Heavy Shield",           // shield
    "Bazooka",                // weapon
    null                      // hasBackpack
);
```

Fluent Builder Construction

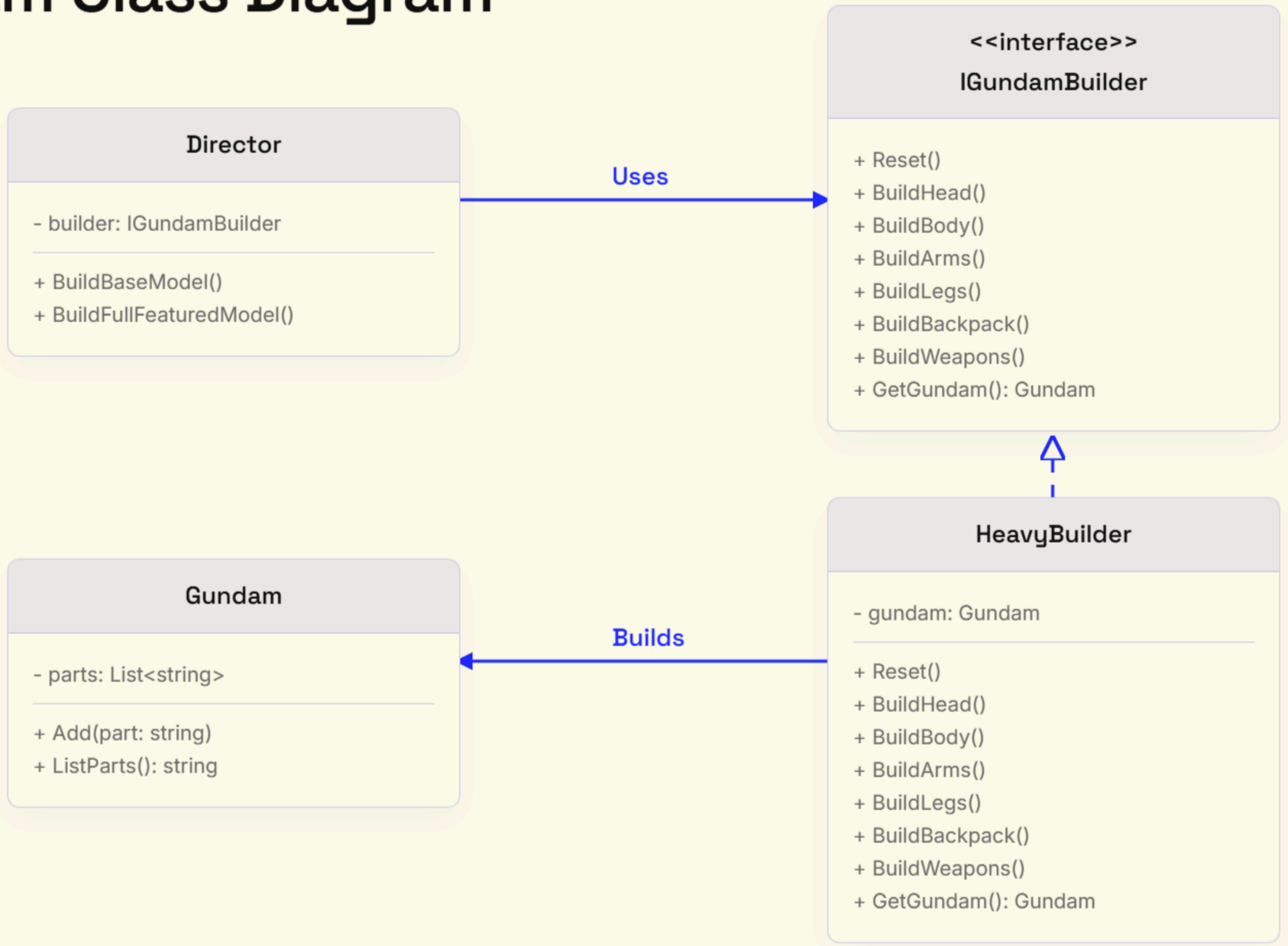
We use a Builder to construct the house step-by-step. The code is self-documenting and we only invoke the steps we actually need.

```
// Clean, readable, and intent-revealing fluent interface.
HeavyBuilder gundamBuilder = new HeavyBuilder();
Gundam heavyGundam = gundamBuilder
    .BuildHead()
    .BuildBody()
    .BuildArms()
    .BuildLegs()
    .BuildBackpack()
    .BuildWeapons()
    .Build();
```

Three Unique Builds



Gundam Class Diagram



Product

```
// The complex product we want to build
public class Gundam
{
    private readonly List<string> _parts = new();

    public void Add(string part)
    {
        _parts.Add(part);
    }

    public string ListParts()
    {
        return "Gundam Parts:\n- " + string.Join("\n- ", _parts);
    }
}
```

Interface

```
// The builder interface specifying methods for creating the different parts
public interface IGundamBuilder
{
    void Reset();
    void BuildHead();
    void BuildBody();
    void BuildArms();
    void BuildLegs();
    void BuildBackpack();
    void BuildWeapons();
    Gundam GetGundam();
}
```

Concrete Builder

```
// Concrete builders implement steps defined in the interface
public class HeavyBuilder : IGundamBuilder
{
    private Gundam _gundam = new();

    public void Reset() => _gundam = new Gundam();
    public void BuildHead() => _gundam.Add("Heavy Sensor Head");
    public void BuildBody() => _gundam.Add("Heavy Armor Body");
    public void BuildArms() => _gundam.Add("Heavy Arms");
    public void BuildLegs() => _gundam.Add("Reinforced Legs");
    public void BuildBackpack() => _gundam.Add("Heavy Shield");
    public void BuildWeapons() => _gundam.Add("Bazooka");

    public Gundam GetGundam()
    {
        var result = _gundam;
        Reset(); // Builders should be ready to start producing another product
        return result;
    }
}
```

Director

```
// The Director controls the order of the building steps
public class Director
{
    private IGundamBuilder _builder;
    public IGundamBuilder Builder { set => _builder = value; }

    public void BuildBaseModel()
    {
        _builder.BuildHead();
        _builder.BuildBody();
        _builder.BuildArms();
        _builder.BuildLegs();
    }

    public void BuildFullFeaturedModel()
    {
        BuildBaseModel();
        _builder.BuildBackpack();
        _builder.BuildWeapons();
    }
}
```

Usage

```
// Client Code Usage
var director = new Director();
var heavyBuilder = new HeavyBuilder();
director.Builder = heavyBuilder;

director.BuildFullFeaturedModel();
var heavyGundam = heavyBuilder.GetGundam();
Console.WriteLine(heavyGundam.ListParts());
```

Pros & Cons

Pros

- **Step-by-step:** Construct objects iteratively, defer construction steps, or run steps recursively.
- **Reusability:** Reuse the same construction code when building various representations of products.
- **Single Responsibility:** Isolate complex construction code from the business logic of the product.

Cons

- **Complexity:** The overall complexity of the code increases since the pattern requires creating multiple new classes (Builder, ConcreteBuilders, Director).
- **Dependency:** The client must be aware of the specific builder types if they aren't hidden behind a Director.

Thank You

Questions?